Big Data and Machine Learning for Applied Economics

Week 07: Neural Networks and Imbalanced Classes

Eduard F. Martinez Gonzalez

Department of Economics, Universidad Icesi

October 27, 2026

Previously...

- **We opened the boosting's box:** the department first, ethnicity next; the interactions the machine found are, above all, ethnicity within each department (Afro $\times$ department: 2.4 points).
- **The clue:** permuting a census block raised the error between 1.3 and 3.8 points; retraining without it, only 0.3 to 0.4. The blocks stand in for one another: the 37 columns tell few stories, repeated.
- **And where it fails:** 14.9 error in rural areas against 9.3 in town centers. With the map, another explanation with almost the same error (12.1).

What is left in the kit

One model left to open: the **neural network**. And a classification debt we have carried since week 4: what do you do when one class barely shows up in the data?

Problem Set 1 debrief: the ranking

Five submissions, ranked by the test RMSE of the price (millions of pesos)

	Team	Final model	Validation RMSE	Validation scheme	Test RMSE
1	Andrés Martínez and Lina Vélez	three models combined, plus the *twin*'s price	159.1	random	**165.2**
2	Laura Sinisterra	random forest on log(price), with the census by block	164.8	random	174.2
3	Elkin Mola and Danilo Vanoy	regression of log(price) with stratum and distances	181.8	zones	181.8
4	Juan Felipe Vargas	Lasso: location multiplies area, type, and bathrooms	196.4	zones	197.7
5	Iván Villalba	regression with comuna, coordinates, and interactions	179.4	80/20	204.4
	Benchmark: M2, Part 1's regression (area, bathrooms, type, and coordinates)				225.3
	Benchmark: M0, the training mean				355.2

- **How to read it:** RMSE on the 3,819 test listings. *Validation*: what each team promised with its scheme; *test*: what it delivered. All five beat M2.
- **Do they match?** Four out of five, within 10 million (by zones, 0 and 1 off; at random, 6 and 9 below the test). The fifth promised 179 and delivered 204.

Today's question: the course's last model, in classification

Two topics: a debt (imbalanced classes) and the neural network, piece by piece

- **The debt first:** since week 4 we have classified polling stations without asking what happens when one class is **rare**. Today we pay it with the **2022 first round**: four candidates; one wins 54.2 % of the stations and another, 0.1 %.
- **Then, the neural network:** the last model in the kit, assembled piece by piece: the unit, the activation, the layer, and the output. First for the usual question —does Petro win?— and then with **four classes at once**: Petro, Hernández, Fico, or others?
- **The usual closing:** the race against the logistic (week 4) and the trees (week 5). It does not have to win: today the point is understanding how it works.

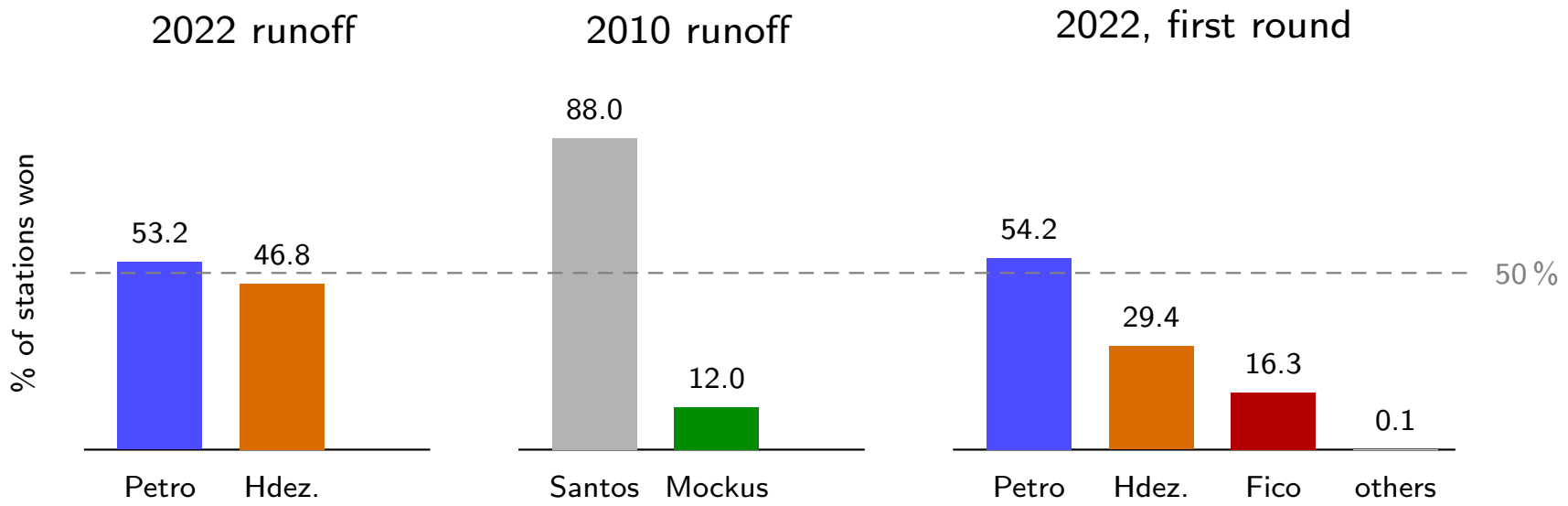
The idea to take home

A neural network is week 4's logistic with variables it manufactures itself. And with a rare class, accuracy misleads: you decide with the right metric and threshold, not by inflating the data.

1. The debt: imbalanced classes
2. The neural network, piece by piece
3. How a network is learned
4. The binary example: does Petro win the station?
5. More than two categories: the first round
6. The race and the closing

Today's y: three elections, three splits

The % of stations each candidate wins; winning means more votes than anyone else



- **How to read it:** each bar is the percentage of stations a candidate wins (2022 in the training set; 2010, the whole table); the dashed line marks 50 %. On the right, the **2022 first round** enters the course today: 11,948 of our 12,000 stations (the rest do not match the runoff or were tied).
- **The split matters:** in the 2022 runoff the classes are nearly even (53/47). In 2010, Santos won 88.0 % of the stations. And the first round has **four classes**: Petro 54.2 %, Hernández 29.4 %, Fico 16.3 %, and *others*, 0.1 %.

Recap (I): the confusion matrix

Before measuring with a rare class: each test station falls into one of four cells

	the model says "Fico"	**says "not Fico"**	total
Fico wins	TP = 204	FN = 199	403
does not	FP = 69	TN = 1,916	1,985
total	273	2,115	$n = 2{,}388$

- **How to read it:** week 4's logistic on the 2,388 first-round test stations, deciding with $\tau = 0.5$. The **rows** are the truth; the **columns**, what the model says. In blue, the hits; in red, the two errors.
- **TP** (true positives): Fico wins and the model says so. **FN** (false negatives): Fico wins and the model misses it. **FP** (false positives): the model says Fico and Fico did not win. **TN**: Fico neither wins nor is called.
- **The totals:** the rows add up what happened (403 Fico stations, 1,985 without Fico); the columns, what the model flagged (273 and 2,115). The measures on the next slide only change the denominator.

Recap (II): four measures, four denominators

The same matrix, read over everyone, over Fico's row, or over the flagged column

Measure	Formula and count	What it asks
Accuracy	$\frac{TP+TN}{n} = \frac{2{,}120}{2{,}388} = 88.8\,\%$	what fraction of *all* does it classify right?
Sensitivity (*recall*)	$\frac{TP}{TP+FN} = \frac{204}{403} = 50.6\,\%$	of the stations Fico *wins* (the row), how many does it find?
Precision	$\frac{TP}{TP+FP} = \frac{204}{273} = 74.7\,\%$	of those it *flags* (the column), how many does Fico win?
Specificity	$\frac{TN}{TN+FP} = \frac{1{,}916}{1{,}985} = 96.5\,\%$	of the stations Fico does *not* win, how many does it rule out?

Why they can tell different stories

Each divides by something else. Accuracy, by everyone: when a class is rare the TNs fill it, and saying "never" is already right a lot. Sensitivity looks only at Fico's row; precision, at the flagged column. Flagging more stations raises sensitivity and usually lowers precision.

Why accuracy misleads when one class is rare

Does Fico win the station? The class shows up at 16.3 % of the training set

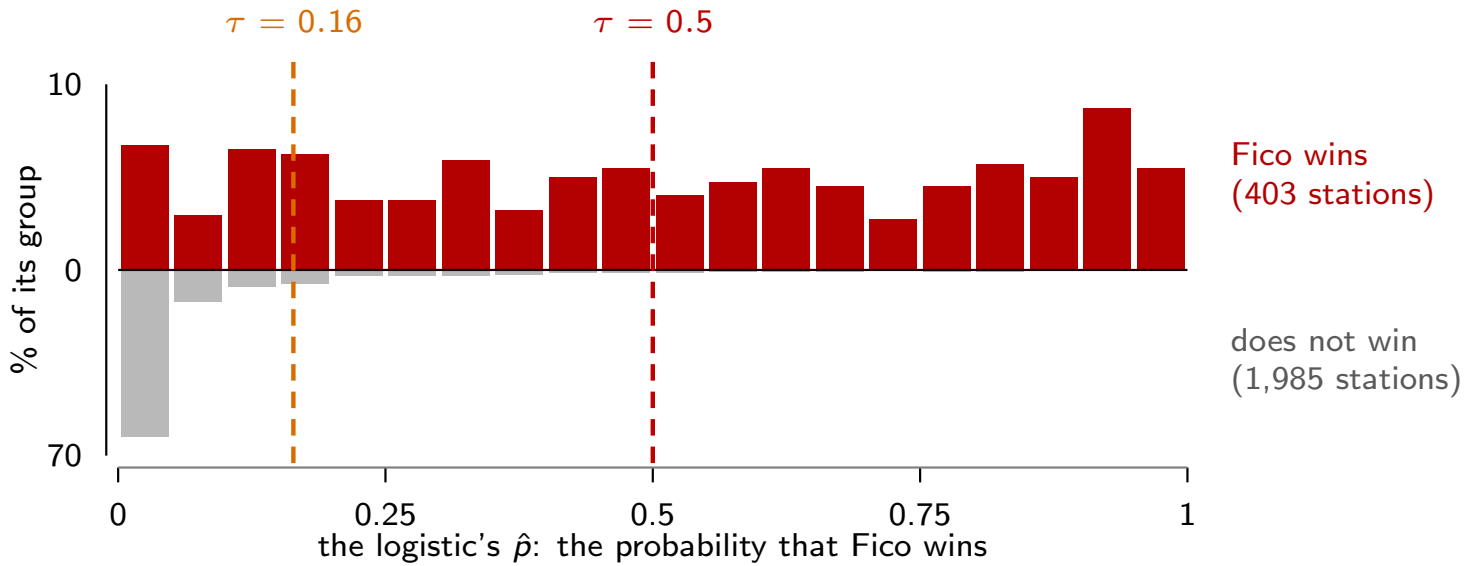
Week 4's benchmark —always call the majority class— here is "Fico never wins": it gets **83.1 %** of the test right without looking at a single datum. **Week 4's logistic** (census + department dummies), with the usual threshold:

	Accuracy (%)	Sensitivity (%)	Precision (%)
"Fico never wins"	83.1	0.0	—
Logistic, $\tau = 0.5$	88.8	50.6	74.7

- **Accuracy sounds splendid** (88.8 %)... but the benchmark that models nothing already got 83.1: with imbalanced classes, the accuracy bar comes for free.
- **Sensitivity tells the uncomfortable truth:** out of every 100 stations Fico wins, the $\tau = 0.5$ rule finds 50.6.
- **The model is not the problem:** it ranks well (AUC 0.902). The **decision rule** is: with a rare class, demanding $\hat{p} \geq 0.5$ is demanding too much.

The first way out (I): what the threshold τ decides

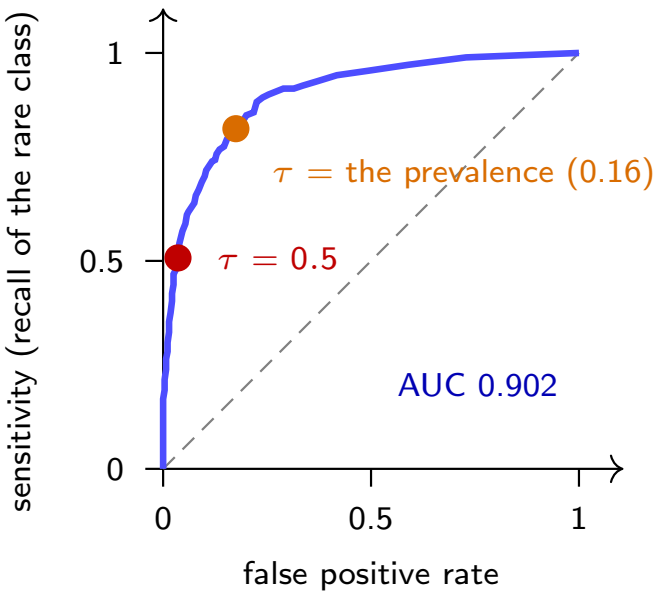
The model delivers a probability; the threshold turns it into a decision



- **How to read it:** on top, the 403 test stations where Fico wins; below, the 1,985 where it does not (each half with its own scale). Each bar is the % of its group with that $\hat{p}$. Whatever lies to the **right** of a line, the model flags as "Fico".
- **With** $\tau = 0.5$ (red line) half of the red ones pass: sensitivity 50.6 % (204 found, 199 missed). Of the gray ones, 3.5 % pass: 69 false alarms.
- **With** $\tau = 0.16$ (orange) 81.9 % of the red ones pass (330) and 17.6 % of the gray ones (350): precision falls to 48.5 %. **The model is the same; the rule changed.**

The first way out (II): the ROC, all thresholds at once

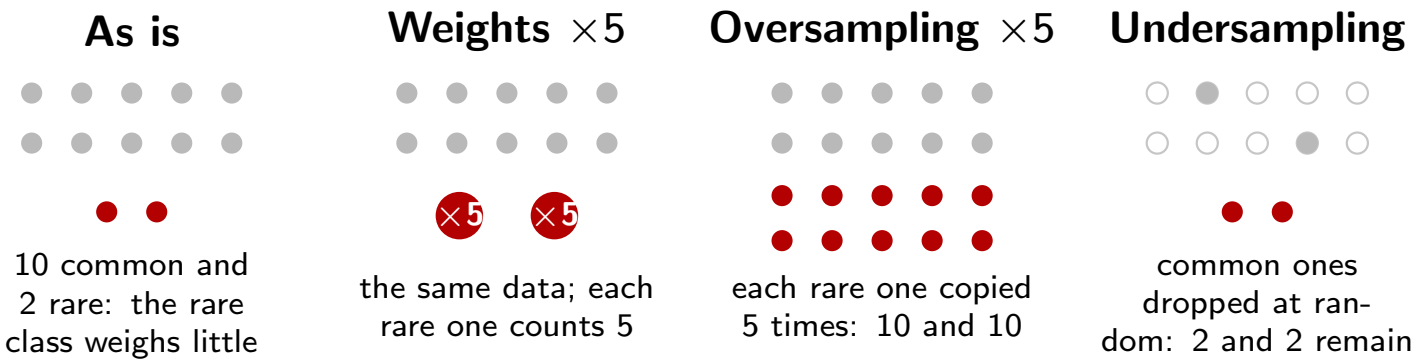
Each point on the curve is a threshold; lowering it moves the point up and to the right



- **How to read it:** the vertical axis is sensitivity (the red ones to the right of τ on the previous slide); the horizontal, the false positive rate, $FP/(FP + TN) = 1-$ specificity (the gray ones to the right). Sweeping τ from 1 to 0 draws the blue curve.
- **The two dots are the same model:** $\tau = 0.5$ (red: 50.6 % and 3.5 %) and $\tau = 0.16$ (orange: 81.9 % and 17.6 %). Moving τ changes neither the curve nor the AUC (0.902): it changes the point where we decide, and the **costs** say which point pays (week 4).

The second way out (I): touch the data or the loss

Three ways for the model to "see" more rare cases; none adds information



- **Weights:** the same data, but in week 4's log-loss each rare station counts five times: $-\sum_i w_i \left[y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i) \right]$, with $w_i = 5$ if Fico wins and $w_i = 1$ otherwise.
- **Oversampling:** copying each rare row five times; each copy adds the same term, so it is *the same* loss as with weights. **Undersampling:** dropping common stations at random until they match the rare ones.
- **All three seek the same thing:** that the rare class weighs more in the fit. And they pay the same price: the model believes the class is more frequent than it is. They are done with the training set only, inside each fold; the test stays as it is.

The second way out (II): what happens in the case

Fico's logistic with each way out, always deciding with $\tau = 0.5$

	AUC	Acc. (%)	Sensit. (%)	Precis. (%)	mean $\hat{p}$ (%)
As is, $\tau = 0.5$	0.902	88.8	50.6	74.7	16.3
As is, $\tau = 0.16$	0.902	82.3	81.9	48.5	16.3
Weights $\times 5$ on the rare	0.904	83.0	82.4	49.7	31.8
Oversampling $\times 5$	0.904	83.0	82.4	49.7	31.8
Undersampling	0.904	82.5	82.6	48.8	32.2

- **The AUC barely moves** ($0.902 \rightarrow 0.904$): weighting or resampling creates no information; the model ranks the stations the same. And look at the weights and oversampling rows: **identical**, as the formula announced: weighting by 5 *is* repeating each rare station 5 times. All three move the decision point, which is what the threshold already did, through the back door.
- **The hidden price:** the mean $\hat{p}$ rises from 16.3 % (what actually happens: 16.9 %) to 31.8 %: the probabilities **stop being probabilities**. If you resample, recalibrate afterwards, and always **inside the fold** (golden rule #3).

The rules of imbalance

What we should have said in week 4; from today on, part of the kit

1. **Look at the split of y before anything else** (describe the target: week 2's rule). With a rare class, accuracy is not the metric: AUC, sensitivity and precision of the rare class, or balanced accuracy (week 4).
2. **The first lever is the threshold,** and the costs set it (week 4). It is free, does not touch the data, and does not damage the probabilities.
3. **Weighting or resampling is equivalent to moving the threshold:** it creates no information and miscalibrates. If you do it, recalibrate afterwards and do it **inside the fold** (golden rule #3).
4. **Some classes cannot be learned:** *others* wins 0.1 % of the stations; with so few cases no fold is enough. You merge it with another class or change the question.

1 The debt: imbalanced classes

2 The neural network, piece by piece

3 How a network is learned

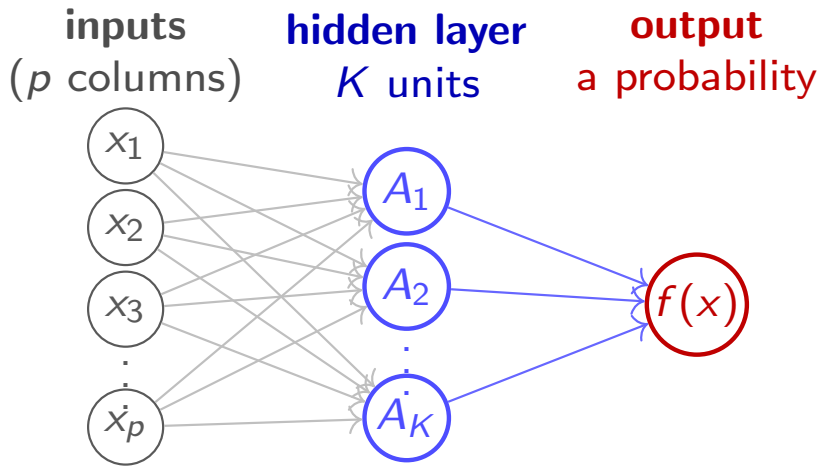
4 The binary example: does Petro win the station?

5 More than two categories: the first round

6 The race and the closing

The neural network, from afar

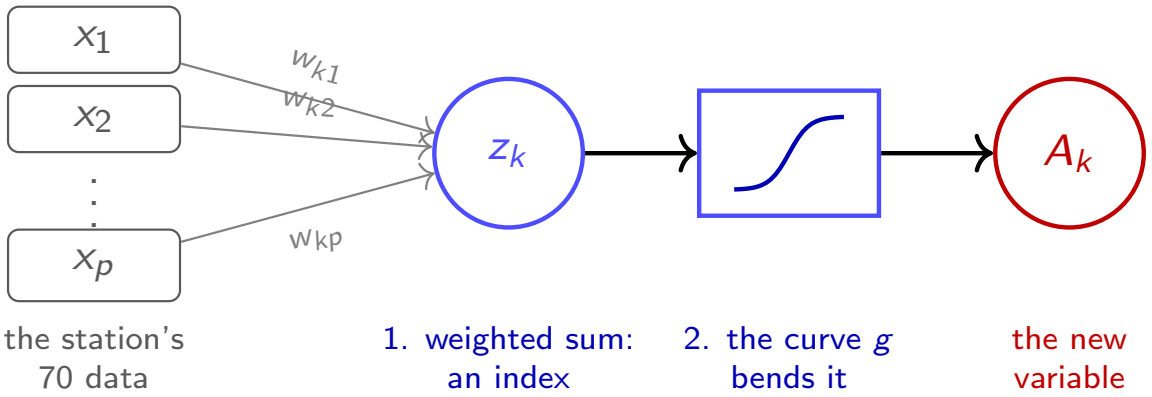
ISL 10.1: a two-stage model that manufactures its own variables. Today we assemble it piece by piece



- **Two stages:** the hidden layer turns the inputs (the 70 census + department columns) into K new variables, $A_1, \ldots, A_K$ (the **activations**); the output turns them into a **probability**: that Petro wins. Everything is learned jointly, looking at y.
- **Today, all in classification** (week 4's y); the same skeleton does regression by changing the output. At the end, four probabilities at once: the first round.
- **The pieces, in order:** the **unit** (what it manufactures), the **layer and the output** (how many go in, how many come out), and the **activation** g (why it bends). Then, how it is learned.

Piece 1 (I): the hidden unit, step by step

Each unit manufactures one new variable: a census index, chosen by y



$$\underbrace{z_k = w_{k0} + w_{k1}x_1 + \cdots + w_{kp}x_p}_{\text{1. a weighted sum: an index}} \qquad \underbrace{A_k = g(z_k)}_{\text{2. the index, bent by the curve } g}$$

- **Step 1:** the sum is a regression's linear predictor: the station's 70 data become a single number, z_k. The weights $w_{k0}, \ldots, w_{kp}$ ($p+1$ per unit) are not handed over: they are learned to predict y.
- **Step 2:** the curve g turns that number into the **activation** A_k. With week 4's sigmoid, A_k stays between 0 and 1: how much the unit *lights up* at that station. Why bending is needed: piece 3.

Piece 1 (II): a variable nobody wrote

One number per station, like one more column of the data set, but manufactured by the network

The four units of the network we are about to choose ($K = 4$), read on the test stations: which census variable each activation resembles.

Unit	Moves with…	Correlation	Lit up ($A_k > 0.5$)
1	the Afro-Colombian population	−0.43	55 % of the stations
2	internet	−0.75	59 %
3	piped gas	−0.64	53 %
4	the population aged 45 to 59	0.38	23 %

- **A reading, not a definition:** each unit mixes the 70 inputs, so none *is* a census variable; the correlation only says which one it resembles. A negative correlation: the unit switches off where that variable is high.

Question for the room: a network *without* a hidden layer —the inputs straight into a sigmoid output—, what model is that?

Piece 1 (II): a variable nobody wrote

One number per station, like one more column of the data set, but manufactured by the network

The four units of the network we are about to choose ($K = 4$), read on the test stations: which census variable each activation resembles.

Unit	Moves with…	Correlation	Lit up ($A_k > 0.5$)
1	the Afro-Colombian population	−0.43	55 % of the stations
2	internet	−0.75	59 %
3	piped gas	−0.64	53 %
4	the population aged 45 to 59	0.38	23 %

- **A reading, not a definition:** each unit mixes the 70 inputs, so none *is* a census variable; the correlation only says which one it resembles. A negative correlation: the unit switches off where that variable is high.

Question for the room: a network *without* a hidden layer —the inputs straight into a sigmoid output—, what model is that?

Week 4's logistic regression: the smallest neural network in the course. What the network adds are the units: indices learned *before* the sigmoid.

Piece 2: how many go in, how many come out

K units in parallel manufacture *K* variables; the output turns them into a probability

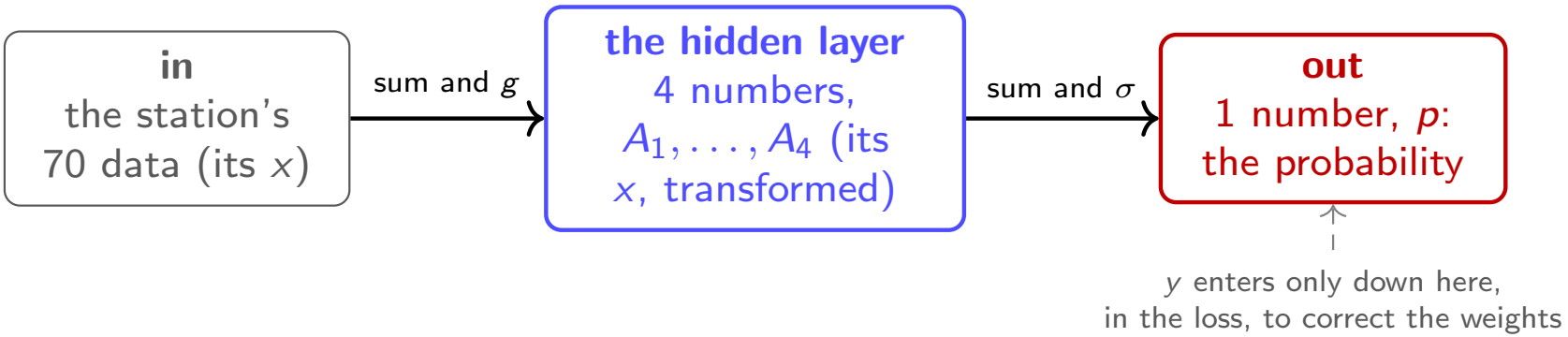
The hidden layer repeats the unit K times, each with its own weights. **The output** is a logistic, but on the K activations, not on the x:

$$p(x) \;=\; \sigma\big(\beta_0 + \beta_1 A_1 + \cdots + \beta_K A_K\big), \qquad \sigma(t) = 1/(1+e^{-t}).$$

- p **go in, 1 comes out:** the 70 columns (census + dummies) go in and **one** probability comes out, as in the logistic. For **regression**, the same network without the sigmoid at the output; for **several classes**, several probabilities (part 5).
- **How many parameters:** each unit has $p+1$ weights and the output, $K+1$. With 70 columns and $K=4$: $4\times 71+5=289$; with $K=16$, 1,153. K is the network's **capacity knob**.
- **What about more layers?** A second layer would do with the A what the first did with the x: weighted sums, bent by g: indices *of the indices*. Today one is enough, the one `nnet` fits.

What travels through the network: two stations, end to end

The chosen network ($K = 4$): each layer reads what the last one built; y stays out



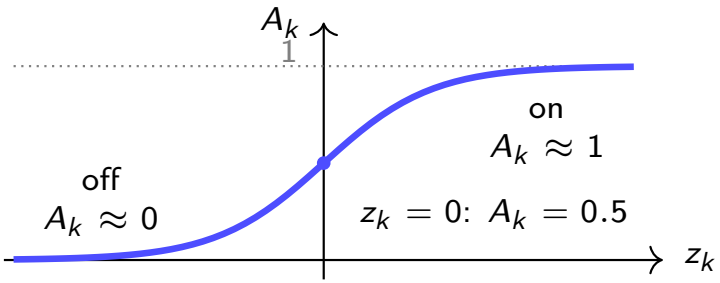
	A_1	A_2	A_3	A_4	p	Did Petro win?
Pedro Grau Arola, Quibdó	0.09	0.79	0.95	0.01	0.99	yes (88.8 %)
Julia Agudelo, Medellín	0.78	0.04	0.02	0.59	0.16	no (40.2 %)
Output weights, β_k	−6.27	−11.59	6.72	−8.40	$\beta_0 = 8.60$	

- **What passes from layer to layer is the station's x, transformed.** The hidden layer reads the 70 data and delivers four numbers, bent by the curve. The output reads those four: in Quibdó, $t = \beta_0 + \beta_1 A_1 + \cdots + \beta_4 A_4 = 5.28$ and $p = \sigma(t) = 0.99$; in Medellín, $t = -1.64$ and $p = 0.16$.
- **What does not pass: y.** Neither y nor a residual travels forward (in week 5's boosting, each tree did read the previous one's residual). y appears only at the end, in the loss, to correct the weights: part 3.

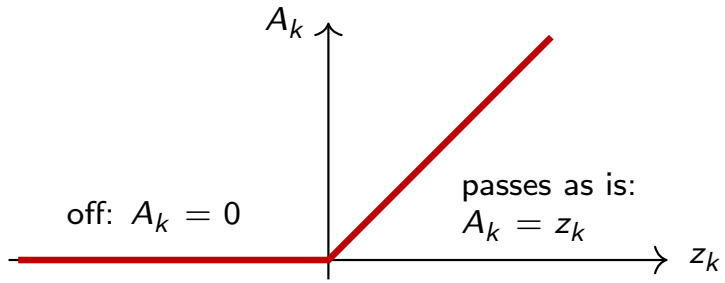
Piece 3 (I): the activation g, a switch

g takes the unit's index z_k and decides how much it lights up at that station

the sigmoid (the one `nnet` uses)



ReLU, $\max(0, z_k)$

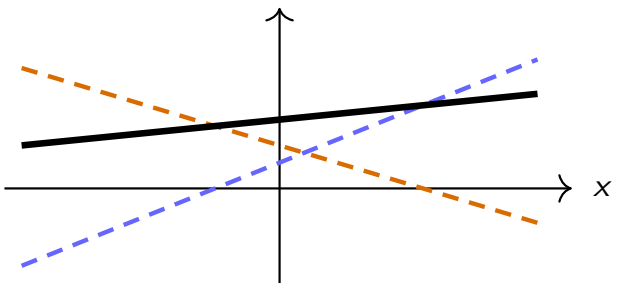


- **The sigmoid** (week 4's): $g(z) = 1/(1 + e^{-z})$. With z_k very negative the unit switches **off** ($A_k \approx 0$); very positive, it switches **on** ($A_k \approx 1$); at $z_k = 0$ it is 0.5. A soft switch.
- **ReLU**, $g(z) = \max(0, z)$: switches off the negative and lets the positive through as is. A kink. It is the one deep networks use, because it is cheap to compute.
- **With the two stations of the previous slide:** the same weights give, in unit 1, $z_1 = -2.37$ in Quibdó, which switches it off ($A_1 = 0.09$), and $z_1 = 1.28$ in Medellín, which switches it on ($A_1 = 0.78$).

Piece 3 (II): why the curve cannot be dropped

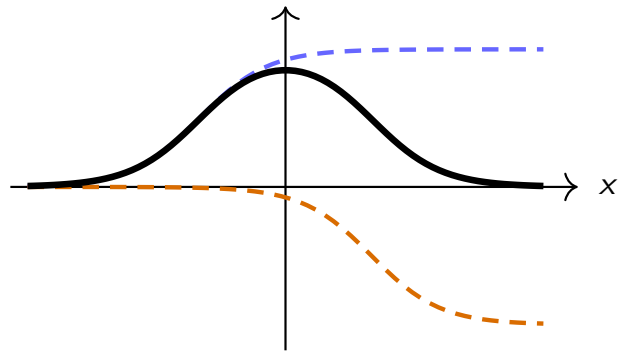
ISL 10.1: without g, adding units gives another line; with g, shapes and interactions appear

without g: adding two lines gives another line



unit 1 + unit 2 = the sum (black)

with g: two sigmoids, $+1$ and -1, make a bump



+unit 1 + −unit 2 = the sum (black)

- **Without** g (or with $g(z) = z$) the network collapses: $\sigma\left(\beta_0 + \sum_k \beta_k(w_{k0} + w_k' x)\right)$ is a sigmoid on a line in x: **week 4's logistic**, however many units (left).
- **With** g, two units already make a *bump* no line can make (right); with more units, more bumps: any smooth shape (appendix A3). And interactions: with $g(z) = z^2$, $\frac{1}{4}\left[(x_1 + x_2)^2 - (x_1 - x_2)^2\right] = x_1 x_2$ (ISL 10.6–10.8), as week 5's trees did with a split inside a split.
- **The price:** with many units it also learns the noise; how many, cross-validation decides (part 4).

1 The debt: imbalanced classes

2 The neural network, piece by piece

3 How a network is learned

4 The binary example: does Petro win the station?

5 More than two categories: the first round

6 The race and the closing

The loss: week 4's likelihood, and going down the gradient

ISL 10.7

The criterion is familiar: choose the weights $\theta = (\beta, w_1, \ldots, w_K)$ that minimize week 4's **log-loss**,

$$R(\theta) \;=\; -\sum_{i=1}^{n} \Big[y_i \log p_\theta(x_i) + (1 - y_i) \log \big(1 - p_\theta(x_i)\big) \Big]$$

(for regression it would be the RSS). **What is new:** because the weights are nested and the units are symmetric, R is **not convex**: it has many local minima. Goodbye to the logistic's guarantees.

Gradient descent: start at random weights θ^0 and step downhill,

$$\theta^{t+1} \;=\; \theta^t - \rho \, \nabla R(\theta^t),$$

with ρ the **learning rate**. In large networks, the gradient is computed on mini-batches (stochastic gradient descent, SGD; appendix A2).

- **Two brakes against overfitting** (ISL): penalizing the weights (weight decay = Ridge, week 3) and stopping in time (early stopping; the boosting rounds, week 5): part 4.

Backpropagation: the residual, split up

ISL 10.7.1; the full derivation, in appendix A1

With the log-loss and the sigmoid output, the gradient of one observation **starts with the residual** $y_i - p(x_i)$: the logistic's first-order condition, again (week 4):

$$\frac{\partial R_i}{\partial \beta_k} = -\big(y_i - p(x_i)\big)\, A_{ik}, \qquad \frac{\partial R_i}{\partial w_{kj}} = -\big(y_i - p(x_i)\big)\, \beta_k\, g'(z_{ik})\, x_{ij}.$$

- **Differentiating splits the residual** among the weights: first among the units, by how much each activated; then among the inputs, through each unit. Hence the name: the error travels from the output backwards.
- It is the boosting residual again (week 5): there each new tree fitted $y - \hat{p}$; here each step moves each weight in proportion to its share of that residual.
- Nobody derives it by hand: the packages do it alone (automatic differentiation). The full gradient costs one forward pass and one backward pass.

1 The debt: imbalanced classes

2 The neural network, piece by piece

3 How a network is learned

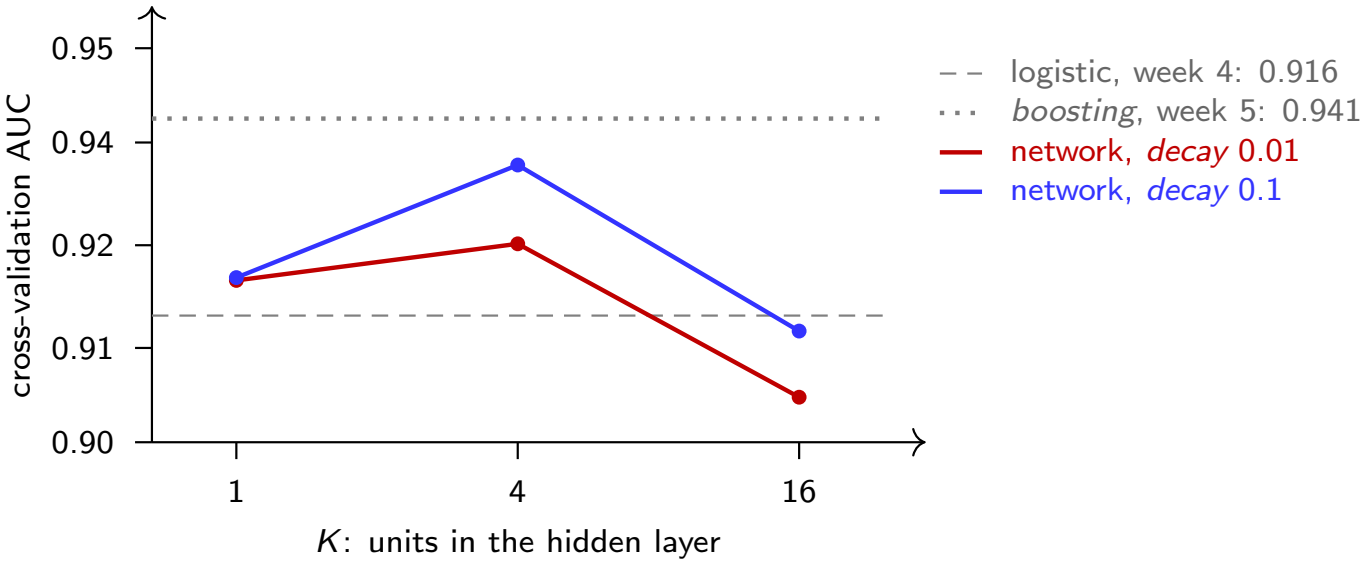
4 The binary example: does Petro win the station?

5 More than two categories: the first round

6 The race and the closing

How many units? The knob K, with cross-validation

Week 3's ten folds; the AUC as the yardstick, as in week 4



- **How to read it:** each dot is a network (census + department) judged with the ten folds; the vertical axis is the cross-validation AUC. Red: decay 0.01; blue: decay 0.1 (weight decay is Ridge on the weights, week 3). The gray lines: the logistic (0.916) and the boosting (0.941).
- **Cross-validation picks** $K = 4$ and decay 0.1, with CV AUC 0.935: just 289 weights for 9,602 stations. **At** $K = 16$ **the AUC falls on both curves:** more units do not always pay, and cross-validation sees it.

The chosen network, on the test set

Ranking, deciding, and quantifying: week 4's three readings

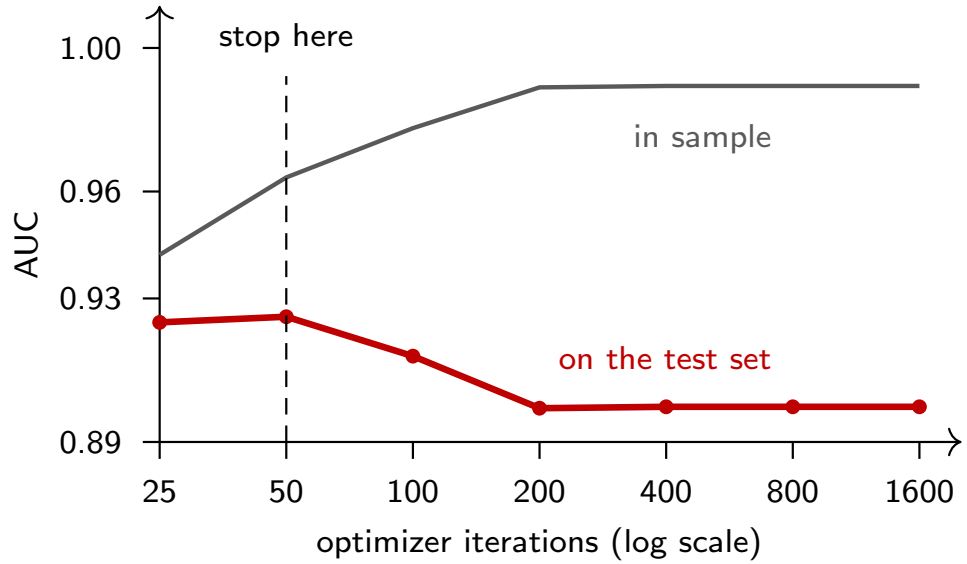
With $\tau = 0.5$, the confusion matrix on the 2,398 test stations:

	Predicted: Petro wins	Predicted: Hernández wins
Actual: Petro wins	TP = 1,095	FN = 173
Actual: Hernández wins	FP = 166	TN = 964

- **It ranks:** test AUC 0.931. **It decides:** accuracy 85.9 % at $\tau = 0.5$. **It quantifies:** Brier 0.104.
- **The threshold is still a decision:** here the classes are nearly even and 0.5 is reasonable; with the campaign's costs, week 4 moved it to 0.83. And if the class were rare, part 1.
- **Two knobs remain** that the non-convex loss gives us: when to stop and where to start. The next two slides.

Stopping in time (early stopping): the third knob

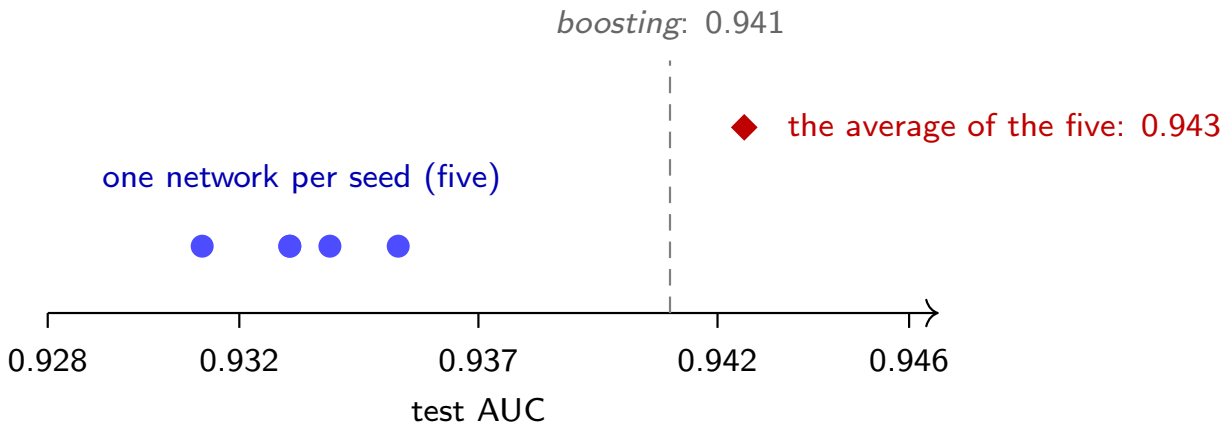
The 16-unit network without weight decay, stopped after 25, 50, . . . , 1,600 iterations



- **How to read it:** the in-sample AUC (gray) and the test AUC (red) of the same unpenalized network, stopped at each point on the axis. In sample it only rises (up to 0.989); on the test set it peaks at 50 iterations (0.925) and then falls (0.900).
- **Early stopping** is the boosting rounds knob (week 5): learning less is a brake. In practice you stop with a validation set, never with the test set.

The seed is a hyperparameter too

The chosen network, trained with five different seeds: each one's AUC and their average's



- **Same data, same network, other initial weights:** the test AUC barely moves (from 0.931 to 0.935), but they are not the same function: since the loss is not convex, each seed ends in another local minimum.
- **Averaging the five networks' probabilities** leaves 0.943, better than any single one. It is the forest again (week 5): averaging models that err differently. You report the spread across seeds, never the best one.

1 The debt: imbalanced classes

2 The neural network, piece by piece

3 How a network is learned

4 The binary example: does Petro win the station?

5 More than two categories: the first round

6 The race and the closing

When several probabilities come out: the softmax

ISL 10.2: the same network, with one output per class; the probabilities sum to one

Only the output changes. For M classes, the network produces M numbers $t_1(x), \ldots, t_M(x)$ —each one its own regression on the A_k— and the **softmax** turns them into probabilities:

$$p_m(x) = \frac{e^{t_m(x)}}{\sum_{\ell=1}^{M} e^{t_\ell(x)}}, \qquad p_1(x) + \cdots + p_M(x) = 1.$$

- **With $M = 2$ it is the usual sigmoid.** The loss also generalizes: the **cross-entropy**, $-\sum_i \log p_{m_i}(x_i)$, with m_i the true class of i (the log-loss with M classes).
- **How many in, how many out:** the 70 columns go in; **four** probabilities come out (Petro, Hernández, Fico, others). The output goes from $K + 1$ weights to $M(K + 1)$: 304 in total.
- **You decide as always:** the class with the highest probability, or the one the costs say. The logistic also has this version (the multinomial, week 4's appendix); the network adds the units.

The first round: four classes, and the imbalance in action

The network with the binary knobs ($K = 4$, decay 0.1); the test confusion matrix

Actual ↓ Predicted →	Petro	Hernández	Fico	others	Recall (%)
Petro	1,129	94	61	0	87.9
Hernández	62	606	32	0	86.6
Fico	120	43	240	0	59.6
others	1	0	0	0	0.0

- **The benchmark first:** saying "always Petro" gets 53.8 % of the test right; the network, **82.7 %**.
- **Part 1's imbalance, in action:** the network **never** predicts *others* (the 0.1 % class; recall 0.0 %), and it finds Fico only 59.6 times out of 100. Overall accuracy hides it; the matrix and the per-class recall show it.

One station, four probabilities

The most contested station on the test set, according to the network

Vereda La Sierra (Córdoba), in the first round:

	Petro	Hernández	Fico	others
$\hat{p}$ of winning the station (%)	33.5	33.0	32.9	0.6

- **They sum to 100:** that is what the softmax buys. At that station, Petro won.
- **To decide** you can take the largest… or weight each error by its cost, as in week 4: writing off one of your own candidate's stations does not cost the same as writing off a rival's.
- **The same output serves** three or ten candidates, employment sectors, crop types: any categorical y.

1 The debt: imbalanced classes

2 The neural network, piece by piece

3 How a network is learned

4 The binary example: does Petro win the station?

5 More than two categories: the first round

6 The race and the closing

The classification race: everyone against everyone

Does Petro win the station? The test AUC of the whole course, with the same split

Model (census + department)	Test AUC	Week
Always say "Petro wins"	0.500	4
Logistic regression	0.916	4
k-NN on the map	0.945	4
Tree	0.896	5
Random forest	0.943	5
Boosting	0.941	5
Neural network	0.931	today
Average of five networks	0.943	today

- **It did not have to win:** today the point was understanding the algorithm. The network alone lands between the logistic and the trees; averaging the five seeds lifts it to 0.943 and catches the forest.
- **When the network pays:** with **structure** the columns do not carry (images, text), with **millions** of data points, or to **borrow** a representation learned elsewhere. On tables like ours, tree ensembles rule (Grinsztajn et al., 2022).

Recap

1. **If a class is rare, accuracy misleads:** look at the split of y, judge with the AUC and the rare class's sensitivity/precision, and decide with the threshold and the costs.
2. **Weighting or resampling creates no information:** it moves the decision point (what the threshold already did) and miscalibrates the probabilities; if you do it, recalibrate, and inside the fold.
3. **A neural network is the logistic with manufactured variables:** each unit is a regression bent by g; without g, everything collapses to the logistic.
4. **It is learned by going down the log-loss gradient** (backpropagation: the residual $y - \hat{p}$, split up) and regularized with the usual knobs: weight decay (Ridge), early stopping, averaging seeds.
5. **More than two classes:** the softmax produces one probability per class and they sum to one; the very rare class does not learn on its own.
6. **On our table the network did not dethrone the trees:** it pays with images, text, or millions of data points. Understanding the algorithm was today's task.

Getting ready for next class...

To get the most out of next session

Tuesday, November 3 is the papers session. Six 15-minute presentations, one per field of economics, with the papers chosen in session 2. Script: the question and the decision; the data and the unit; the benchmark; how it is evaluated out of sample and where it fails; the main result; which method of the course it is and what we would have done differently.

Project Deliverable 1, the same day: the one-page idea (the question and the decision motivating it, the unit and the y, the data and what would be known at prediction time), sent before class, and a 5-minute pitch. Worth 10 of the project's 100 points.

Problem Set 2: due Tuesday, November 10 at 11:59 p.m. With today's session you have everything you need (Part 3 is the network against the forest); there is no class on November 10, but there are office hours.

Reading for the close: Athey and Imbens (2019), sections 1 to 3, for the causal bridge that ends session 8.

R application: two scripts in `applications/week-07`: today's application (a tiny network painted from the inside —weights, activation, and the step-by-step calibration—, two first-round categories, and a large network against a random forest) and the full replication of the deck's figures.

Essential references

- James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An Introduction to Statistical Learning* (2nd ed.). Springer. Ch. 10. [ISL]
- Grinsztajn, L., Oyallon, E., & Varoquaux, G. (2022). Why Do Tree-Based Models Still Outperform Deep Learning on Typical Tabular Data? *NeurIPS Datasets and Benchmarks*.
- Dell, M. (2025). Deep Learning for Economists. *Journal of Economic Literature*, 63(1), 5–58.
- LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep Learning. *Nature*, 521, 436–444.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. Chs. 6 to 8.
- King, G., & Zeng, L. (2001). Logistic Regression in Rare Events Data. *Political Analysis*, 9(2), 137–163.
- Athey, S., & Imbens, G. (2019). Machine Learning Methods That Economists Should Know About. *Annual Review of Economics*, 11, 685–725.
- Gelvez, J. D., Cardiles, A., Martínez-González, E. F., & Muñoz, M. (2026). How Predictable Is an Election? Working paper. [Where the case data come from.]

Appendix: digging deeper

A1. Backpropagation: the one-hidden-layer derivation

The model (regression, one observation's squared loss):

$$z_k = w_{k0} + \sum_j w_{kj}\, x_j, \quad A_k = g(z_k), \quad f = \beta_0 + \sum_k \beta_k A_k, \quad R = \tfrac{1}{2}\,(y - f)^2.$$

1. The output layer: $\dfrac{\partial R}{\partial \beta_k} = -(y - f)\, A_k$: the residual, split by how much each unit activated.

2. The hidden layer, by the chain rule through f and g:

$$\frac{\partial R}{\partial w_{kj}} = \frac{\partial R}{\partial f} \cdot \frac{\partial f}{\partial A_k} \cdot \frac{\partial A_k}{\partial z_k} \cdot \frac{\partial z_k}{\partial w_{kj}} = -(y - f)\, \beta_k\, g'(z_k)\, x_j.$$

3. Backward: unit k's "error" is $\delta_k = -(y - f)\, \beta_k\, g'(z_k)$, and all its gradients are $\delta_k\, x_j$. With more layers, the same step repeats layer by layer, from the output toward the input, reusing the δ's already computed.

With ReLU, $g'(z) = \mathbb{I}\{z > 0\}$: the error only flows through active units. With the sigmoid, $g'(z) = g(z)(1 - g(z))$, nearly zero in the tails: the gradient vanishes.

A2. Stochastic gradient descent, momentum, and Adam

Mini-batches: instead of summing the gradient over the n data points, a batch of 32 to 256 is used at each step; an **epoch** is one full pass. Noisy but cheap, and the noise helps escape bad valleys.
Momentum: move in the direction of a moving average of the gradient,

$$m_t = \beta_1\, m_{t-1} + (1-\beta_1)\, g_t, \qquad \theta_t = \theta_{t-1} - \rho\, m_t.$$

Adam (Kingma and Ba, 2015): it additionally scales each parameter's step by its own volatility,

$$v_t = \beta_2\, v_{t-1} + (1-\beta_2)\, g_t^2, \qquad \theta_t = \theta_{t-1} - \rho\, \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon},$$

with $\hat{m}_t = m_t/(1-\beta_1^t)$ and $\hat{v}_t = v_t/(1-\beta_2^t)$. The usual values: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\rho \approx 10^{-3}$.

- `nnet`, the application's package, uses no batches: it optimizes with BFGS (a quasi-Newton) over all the data, the sensible choice for networks of hundreds of weights. Million-weight networks use Adam.

A3. Universal approximation: what it says and what it does not

The theorem (Cybenko, 1989; Hornik, 1991). If g is not a polynomial (the sigmoid, ReLU), for every continuous f on a compact set and every $\varepsilon > 0$ there exist K and weights such that

$$\sup_x \left| f(x) - \sum_{k=1}^{K} \beta_k \, g\left(w_k' x + b_k\right) \right| < \varepsilon.$$

- **It is a density result,** like Weierstrass's for polynomials. It does not say how many units are needed (K can grow exponentially with p), nor that gradient descent finds them, nor anything about the test error.
- **Depth:** there are functions a deep network represents with few units while a one-layer one needs exponentially many (Telgarsky, 2016): composition reuses pieces.
- **The week 2 analogy:** splines also approximate any smooth function with fixed knots; the network learns *where* to put the knots and on which combinations of variables.

Appendix references and others cited

- Rumelhart, D., Hinton, G., & Williams, R. (1986). Learning Representations by Back-Propagating Errors. *Nature*, 323, 533–536. [A1]
- Kingma, D., & Ba, J. (2015). Adam: A Method for Stochastic Optimization. *ICLR*. [A2]
- Cybenko, G. (1989). Approximation by Superpositions of a Sigmoidal Function. *Mathematics of Control, Signals and Systems*, 2, 303–314. [A3]
- Hornik, K. (1991). Approximation Capabilities of Multilayer Feedforward Networks. *Neural Networks*, 4(2), 251–257. [A3]
- Telgarsky, M. (2016). Benefits of Depth in Neural Networks. *COLT*. [A3]